BLUEPRINT D'UN PROJET PORTFOLIO

ShopPulse

Une plateforme complète de Data Engineering pour analyser un e-commerce en temps réel

Ce guide présente l'objectif, l'architecture, les données, le pipeline, les indicateurs, les contrôles qualité et le plan de réalisation, dans un langage accessible.

STACK PRINCIPALE	RÉSULTAT
Docker | PostgreSQL | Kafka | Kestra | GCS | Spark | BigQuery | dbt | Terraform	Un pipeline fonctionnel qui transforme l'activité de la boutique en tableaux de bord fiables.

Version française - Guide prêt pour la mise en œuvre

ORIENTATION

Comment utiliser ce guide

Lisez d'abord les pages 3 à 5 pour comprendre le problème métier et le flux complet. Utilisez ensuite les autres pages comme référence de construction.

03 Objectif du projet
Problème, utilisateurs et valeur attendue

04 Architecture du système
Rôle de chaque technologie

05 Parcours d'une cliente
Un événement concret dans tout le pipeline

06 Sources et données
Tables, événements, propriétaires et granularité

07 Streaming avec Kafka
Topics, messages, consommateurs et garanties

08 Data lake et Spark
Historique brut, nettoyage, partitions et sessions

09 BigQuery et dbt
Entrepôt, faits, dimensions et tests

10 Indicateurs et dashboard
Revenus, conversion, produits et paiements

11 Orchestration et fiabilité
Planification, reprises, qualité et observation

12 Infrastructure et coûts
Docker, Terraform, sécurité et maîtrise des dépenses

13 Plan de réalisation
Cinq phases du MVP au portfolio

14 Dépôt et workflow
Dossiers, environnements et CI

15 Démo et définition de terminé
Comment prouver que le projet fonctionne

16 Glossaire et décisions
Termes clés et choix recommandés

Règle de conception la plus importante

N'ajoutez pas un outil uniquement parce qu'il apparaît dans une formation. Chaque composant doit résoudre un problème précis. La première version doit fonctionner de bout en bout avant d'ajouter le streaming et le traitement distribué.

01 / OBJECTIF

Pourquoi ShopPulse doit exister

ShopPulse fournit à une boutique en ligne une vue unique et fiable des ventes, du comportement client, des produits et de la santé des paiements.

Objectif métier

Aider un responsable à répondre rapidement : combien avons-nous vendu ? Quels produits fonctionnent ? Où perdons-nous les visiteurs ? Les paiements échouent-ils davantage ?

Objectif d'apprentissage

Prouver que vous savez concevoir, construire, tester, expliquer et exploiter une plateforme moderne depuis l'ingestion jusqu'au tableau de bord.

La situation

Une boutique en ligne produit deux types d'informations. La base opérationnelle enregistre les clients, produits, commandes et paiements. Le site produit un flux beaucoup plus rapide de vues, recherches, actions de panier et tentatives de paiement.

Ces données sont utiles séparément, mais les décisions exigent de les combiner, les nettoyer et les interpréter avec des règles cohérentes. ShopPulse réalise ce travail tout en conservant l'historique brut pour pouvoir rejouer les traitements.

Utilisateurs principaux

Utilisateur	Besoin	Réponse de ShopPulse
Responsable boutique	Comprendre rapidement la performance	Revenus, commandes, clients, conversion, tendances
Analyste marketing	Comprendre l'acquisition et le comportement	Sessions, canaux, vues, abandons du funnel
Équipe opérations	Détecter les problèmes de commande ou paiement	Échecs, annulations, fraîcheur et alertes
Data Engineer	Exploiter un pipeline fiable et explicable	Planification, logs, tests, lineage et données rejouables

Critère de réussite

Un responsable ouvre un seul dashboard et peut faire confiance à des chiffres frais, traçables, testés et reproductibles depuis les données brutes.

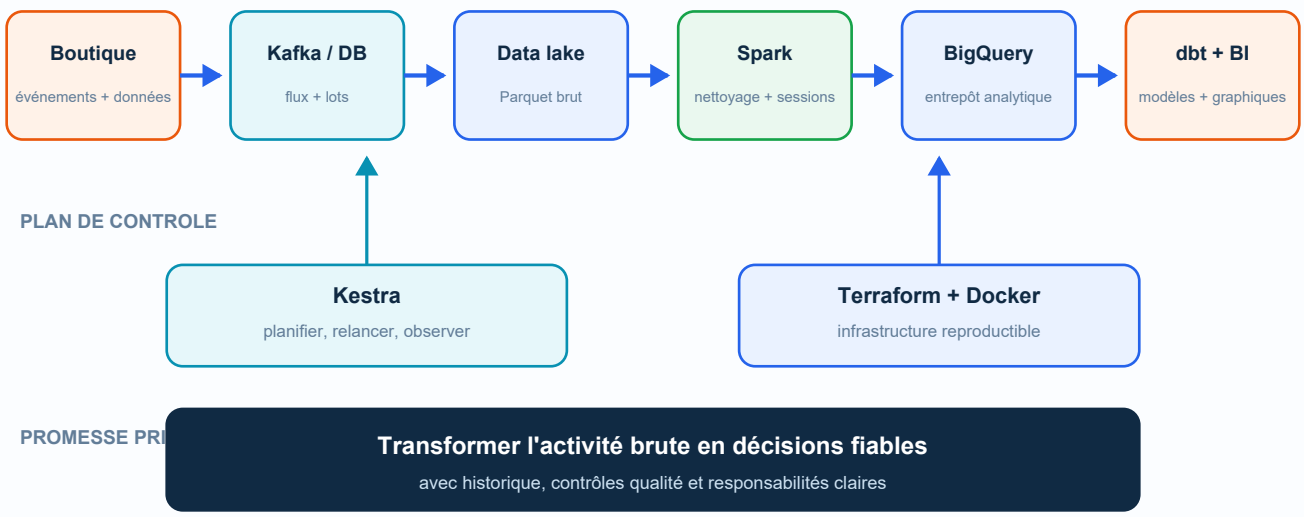
Questions auxquelles le système doit répondre

- Quel chiffre d'affaires avons-nous réalisé aujourd'hui, cette semaine et ce mois-ci ?
- Quels produits sont souvent consultés mais rarement achetés ?
- Quel pourcentage des sessions atteint le panier, le paiement et l'achat ?
- Quels moyens de paiement présentent le plus d'échecs ?
- Combien de temps faut-il pour que les données apparaissent sur le dashboard ?

02 / ARCHITECTURE

Le pipeline complet du projet

La plateforme possède un plan de données qui déplace l'information et un plan de contrôle qui planifie, configure et observe le travail.



Composant	Fonction	Pourquoi il est utile
PostgreSQL	Source opérationnelle	Stocke clients, produits, commandes et paiements
Kafka	Transport streaming	Déplace les événements sans coupler producteurs et consommateurs
Data lake GCS	Historique brut	Conserve des fichiers peu coûteux, partitionnés et rejouables
Spark	Traitement d'événements	Déduplique, valide, joint et construit les sessions
BigQuery	Entrepôt analytique	Exécute les requêtes SQL et sert les modèles transformés
dbt	Transformations métier	Crée des modèles documentés et testés
Kestra	Orchestration	Planifie, paramètre, relance et historise les workflows
Docker	Reproductibilité locale	Démarre les mêmes services sur chaque machine
Terraform	Reproductibilité cloud	Crée buckets, datasets et permissions depuis du code

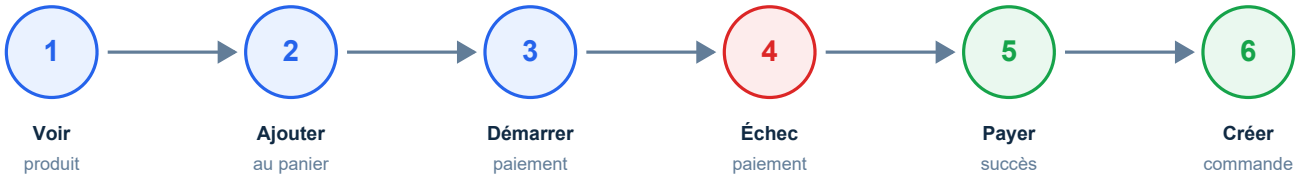
Objectifs de fraîcheur

Les métriques de comportement doivent arriver en 2 à 5 minutes. Les commandes et référentiels peuvent être actualisés chaque heure dans le MVP. Les agrégats quotidiens doivent être complets avant 06:00.

03 / EXEMPLE CONCRET

Suivre une cliente dans toute la plateforme

Un parcours concret rend l'architecture plus simple à comprendre qu'une simple liste d'outils.



Résultat : 1 conversion, 850 $, 1 échec récupéré, parcours de 7 minutes

1. Ada consulte un ordinateur portable

Le site publie un événement product_viewed dans Kafka avec event_id, session_id, product_id, horodatage, canal et appareil.

2. Elle l'ajoute au panier

Un événement de panier est publié. Le consommateur copie les deux messages dans le data lake brut sans modifier leur signification métier.

3. Elle démarre le paiement

L'événement checkout_started permet au modèle de conversion de distinguer l'intention d'achat d'une simple consultation.

4. Son premier paiement échoue

La boutique enregistre la tentative dans PostgreSQL et publie le résultat. Les opérations pourront mesurer les échecs par méthode.

5. Son second paiement réussit

Le paiement réussi et la commande terminée sont enregistrés. La commande est la vérité financière officielle.

6. Le pipeline construit la réponse

Spark regroupe les événements en session. BigQuery reçoit les données propres. dbt joint la session, la commande, le paiement, la cliente et le produit.

Ce que voit finalement le dashboard

Une commande terminée, 850 $ de revenus, un échec de paiement récupéré, trois vues produit, un ajout au panier, un paiement et environ sept minutes entre la première vue et l'achat.

Contrat d'événement - exemple

```
{
  "event_id": "evt_10291",
  "event_type": "product_viewed",
  "customer_id": "cust_421",
  "session_id": "session_702",
  "product_id": "product_91",
  "event_time": "2026-09-07T14:30:00Z",
  "device": "mobile"
}
```

04 / SOURCES

Quelles données entrent dans le système

Le projet sépare volontairement les enregistrements métier officiels des événements de comportement à grand volume.

Système de référence : PostgreSQL

Vérité finale pour les clients, produits, commandes, lignes de commande, paiements et remboursements. L'extraction est incrémentale avec updated_at ou un identifiant croissant.

Flux de comportement : Kafka

Histoire chronologique des sessions, recherches, vues, paniers, paiements et résultats. Les événements sont append-only et possèdent des identifiants stables.

Tables opérationnelles

Table	Granularité	Champs importants
customers	Une ligne par client	customer_id, country, signup_at, acquisition_channel
products	Une ligne par produit	product_id, category_id, name, price, active
orders	Une ligne par commande	order_id, customer_id, ordered_at, status, total_amount
order_items	Une ligne par produit commandé	order_id, product_id, quantity, unit_price
payments	Une ligne par tentative	payment_id, order_id, method, status, amount, attempted_at
refunds	Une ligne par remboursement	refund_id, payment_id, amount, reason, refunded_at

Champs obligatoires d'un événement

Champ	Raison
event_id	Idempotence et suppression des doublons
event_type	Routage et logique des métriques
event_time	Moment où l'action a eu lieu à la source
ingested_at	Moment où la plateforme a reçu l'événement
session_id	Regroupe les actions anonymes ou authentifiées en visite
customer_id	Peut rester vide avant connexion ou paiement
schema_version	Permet de faire évoluer le contrat sans casser les consommateurs

Règle de propriété

Les commandes, paiements et remboursements déterminent le revenu. Kafka explique le comportement et la rapidité, mais ne doit jamais inventer la vérité financière. Cela empêche le double comptage lors d'une nouvelle livraison d'événement.

05 / STREAMING

Conception Kafka et garanties d'événements

Kafka permet à la boutique de publier l'activité une fois, puis à plusieurs consommateurs de la stocker, l'observer ou la traiter à leur rythme.

Topic	Événements	Clé de partition	Rétention
shop.web_events	session_started, searched, product_viewed	session_id	7 jours
shop.cart_events	added_to_cart, removed_from_cart	session_id	7 jours
shop.checkout_events	checkout_started, order_submitted	session_id	14 jours
shop.payment_events	payment_failed, payment_completed	order_id	30 jours
shop.dead_letter	Messages impossibles à valider	topic d'origine	30 jours

Cycle de vie d'un message

1. Le producteur crée event_id avant la publication et valide les champs obligatoires du contrat.
2. Kafka place le message dans une partition. Une clé stable conserve l'ordre pour une session ou une commande.
3. Un consommateur brut écrit le message en Parquet dans une partition par date et heure du data lake.
4. Le consommateur valide son offset seulement après la réussite du stockage. Après un redémarrage, un message peut donc être relu.
5. Spark supprime les doublons avec event_id et conserve l'enregistrement d'ingestion valide le plus récent.
6. Les messages invalides vont dans le topic dead-letter avec la raison de l'erreur.

Garantie : at-least-once

Un message peut être livré plusieurs fois mais ne doit pas disparaître silencieusement. La déduplication en aval rend les répétitions sans danger.

Événements en retard

event_time diffère de ingested_at. Autorisez une fenêtre de retard, recalculez les partitions récentes et mesurez les événements hors fenêtre.

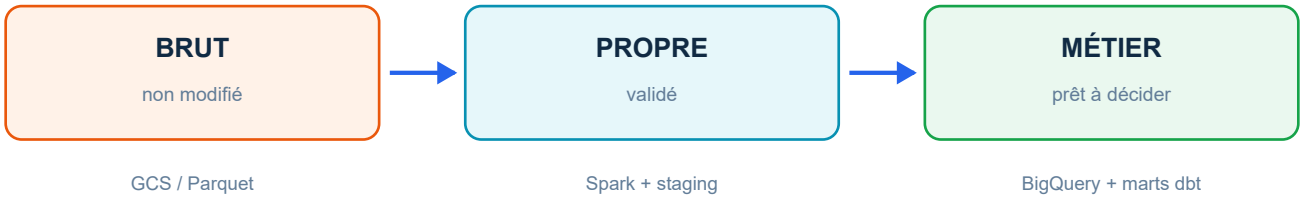
Simplification du MVP

Construisez d'abord le pipeline batch sans Kafka. Ajoutez le streaming à la phase 3, une fois que le dashboard possède déjà une base de données fiable.

06 / STOCKAGE ET TRAITEMENT

Organisation du data lake et jobs Spark

Le data lake préserve l'historique. Spark transforme des fichiers d'événements imparfaits en jeux de données validés et efficaces à interroger.



Organisation recommandée

```
gs://shoppulse-data/
  raw/postgres/orders/load_date=2026-09-07/
  raw/kafka/web_events/event_date=2026-09-07/hour=14/
  quarantine/events/error_date=2026-09-07/
  clean/events/event_date=2026-09-07/
  clean/sessions/session_date=2026-09-07/
```

Responsabilités de Spark

Job	Entrée	Sortie	Logique principale
clean_events	Fichiers Kafka bruts	clean/events	Schéma, horodatage, validation et doublons
build_sessions	Événements propres	clean/sessions	Regrouper par session, durée et étapes du funnel
enrich_events	Événements + produits	clean/enriched_events	Ajouter catégorie, prix et attributs produit
late_event_repair	Partitions brutes récentes	Partitions propres mises à jour	Rejouer une fenêtre glissante pour les retards

Règle de partitionnement

Partitionnez sur les champs qui limitent réellement les lectures, généralement event_date ou order_date. Évitez les partitions à très forte cardinalité comme customer_id, session_id ou event_id.

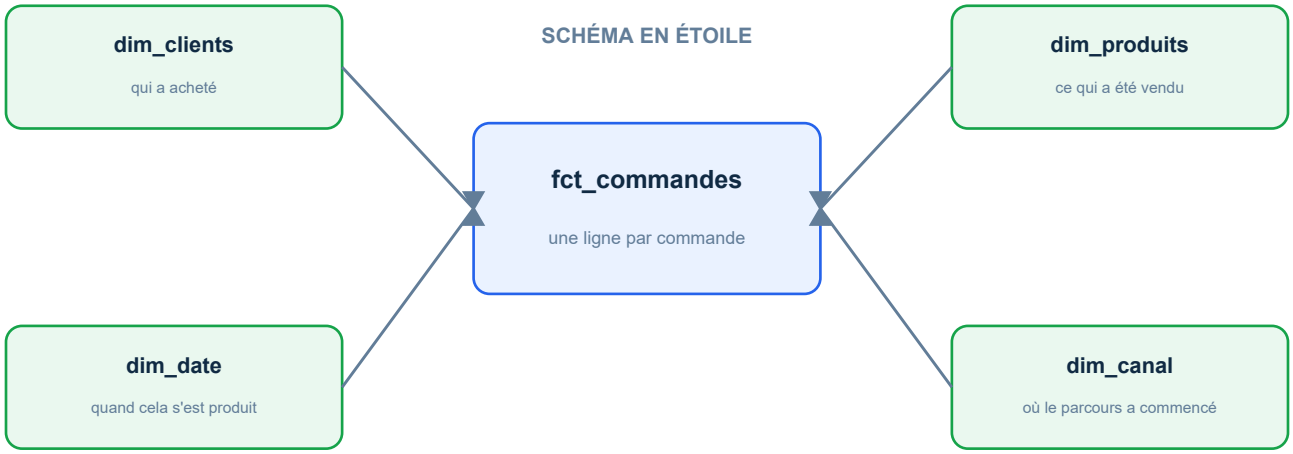
Promesses de la zone brute

- Ne jamais écraser le sens original de la source ; ajouter seulement les métadonnées d'ingestion.
- Conserver assez d'historique pour rejouer une transformation défectueuse.
- Enregistrer source, heure d'extraction, lignes, fichiers et identifiant du run.
- Mettre les données mal formées en quarantaine au lieu de les mélanger aux données valides.

07 / ANALYTICS ENGINEERING

Entrepôt BigQuery et modèles dbt

BigQuery fournit un stockage SQL scalable. dbt transforme les tables brutes en modèles testés qui possèdent un sens métier clair.



Couche	Nommage	Responsabilité	Exemples
Source	raw_*	Données telles que chargées	raw_orders, raw_events
Staging	stg_*	Renommer, typer, dédupliquer, normaliser	stg_orders, stg_payments
Intermédiaire	int_*	Jointures et logique réutilisables	int_order_payments, int_sessions
Core	fct_* / dim_*	Faits et dimensions avec grain déclaré	fct_orders, dim_products
Mart	mart_*	Agrégats prêts pour les dashboards	mart_daily_sales, mart_funnel

Les tests font partie du produit

Test	Erreur évitée
unique + not_null	Deux lignes qui prétendent représenter la même commande
relationships	Une commande liée à un client inexistant
accepted_values	Un statut de paiement inconnu dans les métriques
règle métier	Une commande terminée sans paiement réussi
freshness	Un dashboard qui affiche silencieusement les données d'hier
reconciliation	Un revenu BigQuery différent de la source opérationnelle

Déclarer la granularité

Chaque modèle de faits commence par une phrase : une ligne représente exactement ____. Pour fct_orders : une commande ; fct_payments : une tentative ; fct_sessions : une session web.

08 / RÉSULTAT MÉTIER

Indicateurs et conception du dashboard

Le dashboard rend le travail visible. Chaque graphique doit posséder une formule, un propriétaire, une fréquence de mise à jour et une action métier.



Conversion globale = 900 / 10 000 = 9 %

Indicateur	Définition	Utilité
Chiffre d'affaires	Somme des commandes terminées moins remboursements	Mesure les ventes réalisées
Commandes	Nombre d'identifiants de commandes terminées	Mesure le volume transactionnel
Panier moyen	Chiffre d'affaires / commandes terminées	Suit la valeur moyenne d'achat
Taux de conversion	Sessions acheteuses / sessions éligibles	Mesure l'efficacité du parcours
Abandon panier	Sessions panier sans achat / sessions panier	Montre l'intention perdue
Succès paiement	Tentatives réussies / tentatives finales	Surveille la fiabilité du paiement
Clients récurrents	Clients avec 2+ commandes / clients acheteurs	Mesure la rétention
Fraîcheur	Heure actuelle - dernier événement source réussi	Indique si les chiffres sont à jour

Pages recommandées

Page	Contenu
Vue dirigeant	Revenus, commandes, clients, panier moyen, tendances et géographie
Produits	Top produits, catégories, vue-vers-achat et faibles conversions
Conversion	Sessions, vues, paniers, paiements, achats et abandons par canal
Paiements	Succès, échecs par méthode, échecs récupérés et revenu affecté
Santé des données	Fraîcheur, volumes, tests, retards et dernier run réussi

Exemple de contrat d'indicateur

Le chiffre d'affaires utilise la devise au moment de la transaction, exclut les commandes annulées, soustrait les remboursements terminés, suit le fuseau horaire de la boutique et se met à jour chaque heure.

09 / EXPLOITATION

Workflows Kestra et contrôles de fiabilité

L'orchestration coordonne les dépendances, enregistre l'état, relance sans danger et empêche les mauvaises données d'atteindre les utilisateurs.

Workflow	Fréquence	Étapes	Succès
hourly_orders	Horaire	Extraire changements -> lake -> BigQuery	Watermark avancé et comptes réconciliés
stream_to_lake	Continu	Consommer -> valider -> écrire -> commit	Offsets progressent et peu d'erreurs
event_processing	15 min	Spark clean -> sessions -> chargement	Partitions récentes complètes
dbt_build	Après chargement	Sources -> staging -> core -> marts -> tests	Tous les tests bloquants passent
daily_quality	Chaque jour 06:00	Fraîcheur -> volume -> réconciliation	Aucune anomalie critique

Comportements sûrs

Idempotence

Relancer le même intervalle donne le même résultat final. Utilisez des identifiants stables, des MERGE et des watermarks.

Relances

Relancez les erreurs réseau temporaires avec délai. Ne relancez pas indéfiniment une donnée invalide ou une logique cassée.

Backfills

Acceptez start_date et end_date afin de reconstruire volontairement une période historique.

Barrières qualité

Bloquez la publication si les tests dbt critiques, la réconciliation du revenu ou la fraîcheur échouent.

Observabilité

Enregistrez run_id, début, fin, watermark, lignes lues, écrites, rejetées et détails d'erreur.

Avertissement

Variation de volume non critique ou petit nombre d'événements en retard. Continuez le pipeline mais rendez la condition visible.

Échec bloquant

Champs requis manquants, relations cassées, source périmée ou totaux financiers non réconciliés. Ne publiez pas le mart affecté.

Pourquoi Bruin est optionnel

Bruin recoupe certaines responsabilités de Kestra et dbt. Gardez-le hors du MVP et évaluez-le plus tard dans une branche séparée afin de préserver une architecture simple à expliquer.

10 / FONDATION

Infrastructure, sécurité et maîtrise des coûts

Le projet doit être reproductible sans traiter les identifiants cloud ou les dépenses incontrôlées avec légèreté.

Outil	Responsabilité locale	Responsabilité cloud
Docker Compose	PostgreSQL, Kafka, Kestra, générateur	Ne crée pas les services cloud managés
Terraform	Variables et validation	Bucket GCS, datasets BigQuery, comptes et permissions
Fichiers d'environnement	Configuration non secrète	Project ID, région, noms de dataset et bucket
Secret store	Identifiants hors Git	Compte de service ou workload identity

Minimum de sécurité

- Ne jamais committer mots de passe, clés privées, chaînes de connexion ou identifiants cloud.
- Donner à chaque service seulement les permissions nécessaires ; séparer lecture, écriture et administration.
- Utiliser des noms générés et des clients fictifs. Ne pas placer de données personnelles réelles dans le portfolio.
- Chiffrer le trafic et le stockage avec les options managées, puis documenter chaque frontière.
- Masquer ou exclure noms, emails et adresses des modèles analytiques sauf besoin réel.

Valeurs par défaut pour le coût

Zone	Choix recommandé
BigQuery	Partitionner par date, clusteriser les filtres fréquents, imposer le filtre de partition
GCS	Compresser en Parquet, appliquer un cycle de vie et choisir une rétention explicite
Spark	Utiliser le mode local ; lancer de petits jobs cloud seulement pour la démonstration
Kafka	Exécuter localement avec Docker plutôt que payer un cluster managé
Volume	Assez de lignes pour une histoire réaliste, pas assez pour créer une facture inutile

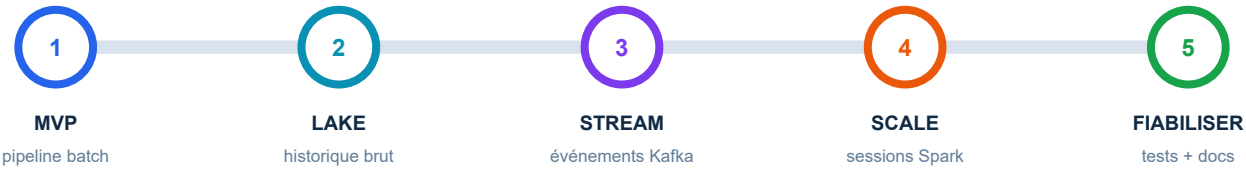
Échelle conseillée

5 000 clients, 200 produits, 20 000 sessions, 2 000 commandes et un flux en direct contrôlable. Suffisant pour des métriques réalistes et assez petit pour rester économique.

11 / PLAN DE LIVRAISON

Construire en cinq phases maîtrisées

Chaque phase se termine par un résultat démontrable. Ne commencez pas la suivante tant que le chemin actuel ne fonctionne pas et n'est pas testé.



Phase	Construction	Preuve de sortie
1 - MVP	Générateur -> PostgreSQL -> Kestra -> BigQuery -> dbt -> dashboard	Une commande démarre les services et produit des ventes quotidiennes fiables
2 - Data lake	Extractions incrémentales en Parquet GCS avant BigQuery	Supprimer une table dérivée et la reconstruire depuis le brut
3 - Streaming	Publier les événements dans Kafka puis les stocker	Vues et paniers apparaissent en moins de cinq minutes
4 - Spark	Nettoyer, dédupliquer, enrichir et créer les sessions	Retards et doublons ne corrompent pas la conversion
5 - Fiabiliser	Terraform, CI, tests, alertes, lineage, docs et démo	Un évaluateur peut déployer, exécuter, comprendre et vérifier

Ordre conseillé dans la phase 1

1. Définir les métriques et les schémas sources avant d'écrire le pipeline.
2. Générer clients, produits, commandes, lignes et tentatives de paiement.
3. Démarrer PostgreSQL et charger un jeu de données déterministe.
4. Créer une extraction Kestra incrémentale avec watermarks updated_at.
5. Charger les tables BigQuery brutes et construire staging, faits, dimensions et ventes quotidiennes.
6. Ajouter les tests puis connecter un dashboard minimal aux marts.
7. Documenter la démonstration depuis une machine propre.

Garde-fou de périmètre

Si la phase 1 n'est pas terminée, n'ajoutez ni Kubernetes, ni machine learning, ni second cloud, ni frontend complexe. Un petit système terminé est plus fort qu'un grand assemblage incomplet.

12 / DÉPÔT

Structure de dossiers et workflow recommandés

Le dépôt doit communiquer clairement les responsabilités : infrastructure, ingestion, traitement, transformations, qualité et documentation ont chacun leur place.

```
shoppulse/
  README.md
  docker-compose.yml
  .env.example
  data-generator/
  ingestion/postgres/
  ingestion/kafka/
  orchestration/kestra/
  processing/spark/
  warehouse/dbt/
  infrastructure/terraform/
  dashboards/
  tests/
  docs/architecture/
```

Environnement	Objectif	Comportement attendu
Local	Développement rapide	Services Docker, seed, Kafka et Spark locaux, sandbox cloud
CI	Vérification automatique	Formatage, tests, dbt compile/tests et validation Terraform
Démo	Présentation portfolio	Petit dataset déterministe et flux streaming contrôlable

Checklist du README

- Problème, utilisateurs visés et captures du dashboard final.
- Schéma d'architecture et explication courte de chaque composant et compromis.
- Prérequis, installation, arrêt, configuration, durée et coût attendus.
- Dictionnaire de données, granularité, définitions des métriques et documentation dbt.
- Procédure de backfill, reprise d'un échec et inspection des événements en quarantaine.
- Limites connues et courte liste d'améliorations futures.

Discipline de branche

Construire une capacité verticale à la fois. Garder les changements faciles à relire et joindre tests et documentation au code.

Reproductibilité

Utiliser des seeds fixes pour les données, des versions de dépendances figées et Terraform pour les ressources cloud.

Contrat d'environnement

Un évaluateur doit pouvoir copier .env.example, ajouter ses propres identifiants, démarrer les services, générer les données, exécuter le pipeline et voir les métriques sans modifier le code source.

13 / PREUVE

Scénario de démo et définition de terminé

Le projet est terminé lorsque vous pouvez prouver son comportement, expliquer ses compromis et récupérer les échecs ordinaires.

Une démonstration portfolio de 8 minutes

1. Présenter le problème : transactions et événements rapides sont séparés.
2. Montrer l'architecture et expliquer chaque composant en une phrase.
3. Démarrer un petit flux et montrer les vues, paniers et paiements dans Kafka.
4. Ouvrir Kestra et montrer le run, les paramètres, les volumes et la barrière qualité.
5. Interroger un modèle de faits BigQuery et montrer sa granularité déclarée.
6. Ouvrir la documentation dbt et montrer tests et lineage de mart_daily_sales.
7. Montrer le dashboard qui évolue et expliquer une observation métier du funnel.
8. Démontrer une reprise : rejouer un intervalle ou montrer un message en quarantaine.

Définition de terminé

Domaine	Le projet est terminé lorsque
Exactitude	Les métriques clés se réconcilient avec les sources et les tests critiques passent
Fraîcheur	Le dashboard affiche sa dernière mise à jour et respecte les objectifs
Fiabilité	Les relances sont sûres, les doublons inoffensifs et les backfills paramétrés
Récupération	Les données dérivées peuvent être reconstruites depuis l'historique brut
Sécurité	Aucun secret ni donnée personnelle réelle n'existe dans Git
Coût	Ressources, partitions, nettoyage et coût de démo sont documentés
Utilisabilité	Un évaluateur peut exécuter le projet uniquement avec le README
Communication	Architecture, modèle, règles, limites et compromis sont expliqués

Test portfolio

Si vous pouvez expliquer comment un message product_viewed devient une métrique, comment les doublons sont gérés et pourquoi le revenu vient des commandes plutôt que des événements, vous comprenez le cœur du projet.

14 / RÉFÉRENCE

Glossaire et décisions recommandées

Utilisez cette page lorsque des termes peu familiers apparaissent pendant la réalisation.

Terme	Signification simple
Batch	Traiter un groupe de données collectées selon un horaire
Stream	Traiter un flux continu peu après l'apparition des événements
Événement	Déclaration immuable qu'une action a eu lieu à un instant précis
Granularité	Ce qu'une ligne représente exactement
Watermark	Dernière position ou heure source traitée avec succès
Idempotent	Peut être exécuté plusieurs fois sans modifier le résultat correct
Partition	Tranche physique ou logique qui limite travail et coût
Fait	Occurrence mesurable : commande ou tentative de paiement
Dimension	Contexte descriptif : client, produit, date ou canal
Data mart	Modèle préparé pour une question métier ou un dashboard
Lineage	Carte des sources et transformations qui produisent un modèle
Backfill	Retraiter volontairement une période historique
Dead-letter	Zone pour les messages qui échouent à la validation
Réconciliation	Comparer les totaux entre systèmes pour détecter pertes ou doublons

Décisions recommandées en un coup d'œil

Décision	Choix recommandé
Première version	MVP batch avant Kafka et Spark
Vérité financière	Commandes, paiements et remboursements PostgreSQL
Vérité comportementale	Événements Kafka versionnés avec event_id stable
Garantie streaming	At-least-once avec déduplication en aval
Format brut	Parquet partitionné avec métadonnées d'ingestion
Orchestration	Kestra ; Bruin reste optionnel pour une évaluation ultérieure
Cloud	GCS et BigQuery créés par Terraform
Données portfolio	Synthétiques, déterministes et sans données personnelles réelles

Prochaine étape pratique

Créer le squelette de la phase 1, définir les tables PostgreSQL et le générateur de données, puis écrire les contrats exacts du revenu et des commandes avant de connecter un service cloud.